EXP 5 MUTEX

```
#include <includes.h>
#include "func.h"

/********** Define Task Priorities ***********/
#define  APP_TASK_START_PRIO              4
#define  APP_TASK0_PRIO                   5
#define  APP_TASK1_PRIO                   6
#define  APP_TASK2_PRIO                   7


/*--------------- AAPLICATION STACKS ---------*/
static  OS_STK       AppTaskStartStk[APP_TASK_STK_SIZE];
static  OS_STK       AppTask0stk[APP_TASK_STK_SIZE];       /* Create the required number of stacks need for every child task*/
static  OS_STK       AppTask1stk[APP_TASK_STK_SIZE];
static  OS_STK       AppTask2stk[APP_TASK_STK_SIZE];


/*-------------LOCAL FUNCTION PROTOTYPES--------------*/

/*--------------- A PARENT TASK (MAIN TASK) ---------*/
static  void  AppTaskStart (void       *p_arg);          /* Main(Parent) Task Function */
static  void  AppTaskCreate(void);                       /* Separate Function To Create Child Task(s) */

/*--------------- CHILDERN TRASKS --------------*/
static  void  AppTask0      (void       *p_arg);
```

```
static  void  AppTask1      (void       *p_arg);
static  void  AppTask2      (void       *p_arg);

OS_EVENT *LCD_Mutex;



int  main (void)
{
   BSP_IntDisAll();                    /* Disable all interrupts until we are ready to accept them */
   OSInit();                           /* Initialize "uC/OS-II, The Real-Time Kernel"              */

   OSTaskCreate(AppTaskStart,                          /* Create the starting task i.e. Main
Task       */
                (void *)0,
                (OS_STK *)&AppTaskStartStk[APP_TASK_STK_SIZE - 1],
                APP_TASK_START_PRIO);

   OSStart();                                          /* Start multitasking (i.e. give control to uC/OS-
II)      */
}



static  void  AppTaskStart (void *p_arg)
{
   unsigned char err;
   p_arg = p_arg;                      /*Just to avoid compiler Warning          */

   BSP_Init();                         /* Initialize BSP functions     */
   InitLCD();                          /* Initialize LCD   */
```

```
    kbdInit();                      /* Initialize Keyboard  */
    ADCInit();                      /* Initialize ADC   */
    LEDInit();                      /* Initialize LED   */
    UartInit(9600);                 /* Initialise the UART*/

    LCD_Mutex = OSMutexCreate(3,&err);     /* Create a Mutex   */

    AppTaskCreate();                /* Create application tasks  (child tasks)       */

    while(DEF_TRUE)
    {
     printf(" \r\nMAIN TASK: Created 3 Tasks. Now going to deep sleep...");
     printf(" \r\n=====================================================\r\n");
     OSTimeDlyHMSM(1, 0, 0, 0);
    }
}

static  void  AppTaskCreate (void)
{
    /* Create User Tasks */
    /* Create Task0 */
    OSTaskCreate(AppTask0,
                 (void *)0,
                 (OS_STK *)&AppTask0stk[APP_TASK_STK_SIZE - 1],
                 APP_TASK0_PRIO);

   /* Create Task1 */
    OSTaskCreate(AppTask1,
```

```
                (void *)0,
                (OS_STK *)&AppTask1stk[APP_TASK_STK_SIZE - 1],
                APP_TASK1_PRIO);

   /* Create Task2 */
   OSTaskCreate(AppTask2,
                (void *)0,
                (OS_STK *)&AppTask2stk[APP_TASK_STK_SIZE - 1],
                APP_TASK2_PRIO);

}
static  void  AppTask0 (void *p_arg)
{
   unsigned char err;
   p_arg = p_arg;                          /*Just to avoid compiler Warning          */

   while(DEF_TRUE)
   {
    /* User Code Here */
       printf("TASK0: Waiting for LCD\n\r");
       OSMutexPend(LCD_Mutex,0,&err);          /*   Wait to acquire Mutex   */
       printf("TASK0: LCD acquired\n\r");
       LCD_display(1,1,"This is TASK0");       /*   Perform operations on LCD   */
       OSTimeDlyHMSM(0,0,15,0);
       OSMutexPost(LCD_Mutex);                 /*   Release Mutex   */
       printf("TASK0: Released LCD\n\r\n\r");
       OSTimeDlyHMSM(0,0,10,0);
```

```
    }
}
static  void  AppTask1 (void *p_arg)
{
    unsigned char err;
    p_arg = p_arg;                              /* Just to avoid compiler Warning        */

    while(DEF_TRUE)
    {
     /* User Code Here */
        printf("TASK1: Waiting for LCD\n\r");
        OSMutexPend(LCD_Mutex,0,&err);                  /*  Wait to acquire Mutex   */
        printf("TASK1: LCD acquired\n\r");
        LCD_display(1,1,"This is TASK1");               /*  Perform operations on LCD   */
        OSTimeDlyHMSM(0,0,5,0);
        OSMutexPost(LCD_Mutex);                         /*  Release Mutex   */
        printf("TASK1: Released LCD\n\r\n\r");
        OSTimeDlyHMSM(0,0,6,0);

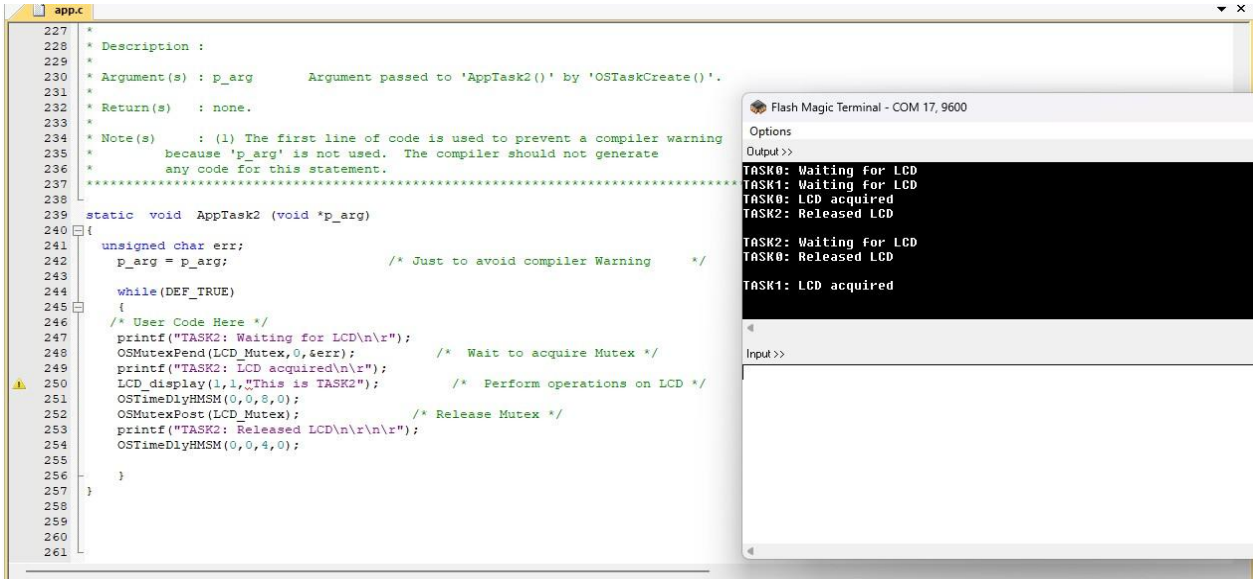
    }
}

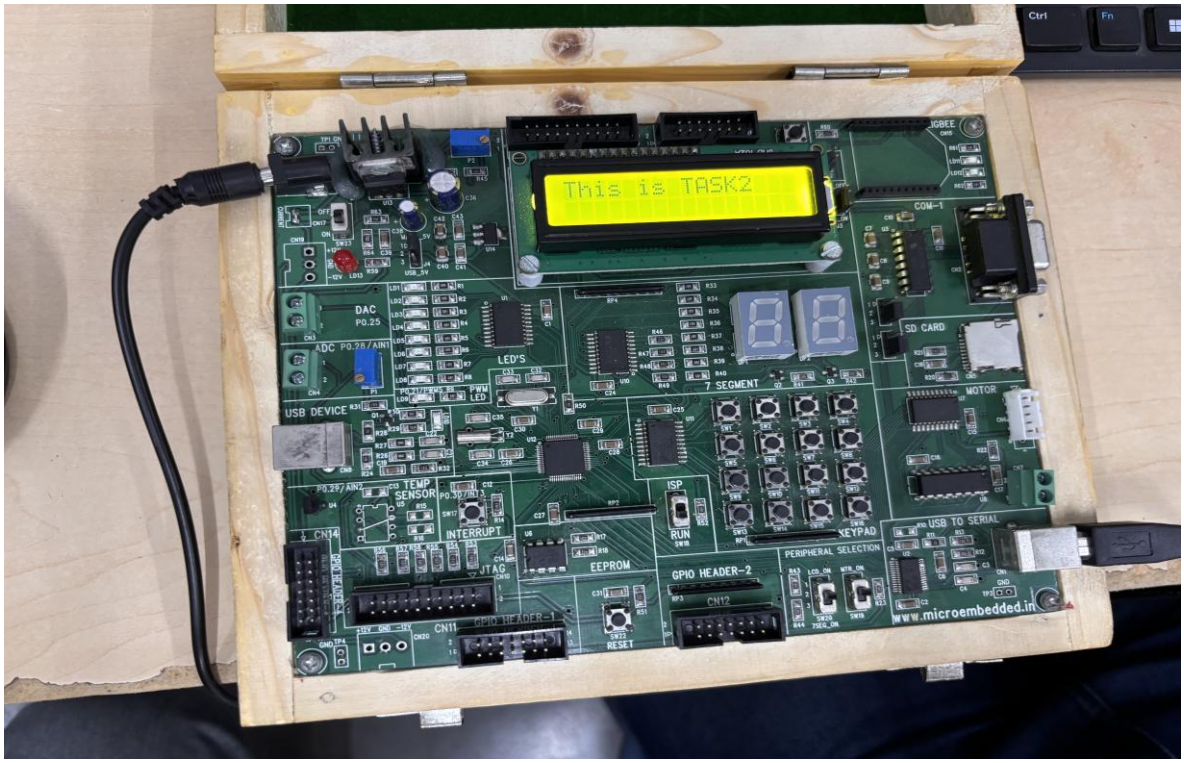
static  void  AppTask2 (void *p_arg)
{
    unsigned char err;
    p_arg = p_arg;                              /* Just to avoid compiler Warning       */

    while(DEF_TRUE)
```

```
  {
   /* User Code Here */
     printf("TASK2: Waiting for LCD\n\r");
     OSMutexPend(LCD_Mutex,0,&err);              /*  Wait to acquire Mutex   */
     printf("TASK2: LCD acquired\n\r");
     LCD_display(1,1,"This is TASK2");           /*  Perform operations on LCD   */
     OSTimeDlyHMSM(0,0,8,0);
     OSMutexPost(LCD_Mutex);                     /*   Release Mutex   */
     printf("TASK2: Released LCD\n\r\n\r");
     OSTimeDlyHMSM(0,0,4,0);

  }
}
```




This is TASK2
DAC
P0.25
ADC P0.28/AIN1
LED'S
PWM LED
USB DEVICE
TEMP SENSOR
P0.29/AIN2
P0.30/AIN3
INTERRUPT
JTAG
EEPROM
ISP
RUN
7 SEGMENT
KEYPAD
SD CARD
MOTOR
COM-1
IGBEE
USB TO SERIAL
PERIPHERAL SELECTION
GPIO HEADER-2
GPIO HEADER-1
RESET
www.microembedded.in